

UNIVERSIDADE DO VALE DO ITAJAÍ
Curso de Ciência da Computação

**SISTEMA DE PROCESSAMENTO PARALELO DE REQUISIÇÕES A UM BANCO
DE DADOS**

José Gabriel Santos Gomes
Matheus Pompeo
Nathan Gustavo Padilha Reichert

Disciplina: Sistemas Operacionais
Professor: Michael Douglas C. Alves
Semestre: 2026/2

Itajaí (SC)
2026

RESUMO

Este trabalho apresenta a implementação de um sistema cliente-servidor que simula um gerenciador de requisições a um banco de dados, aplicando os conceitos de comunicação entre processos (IPC), threads, concorrência e paralelismo. O sistema é composto por dois processos independentes: um processo cliente que envia requisições de INSERT, SELECT, UPDATE e DELETE através de named pipes do Windows, e um processo servidor que as processa em paralelo utilizando um pool de threads implementado com a biblioteca Pthreads. O acesso à tabela compartilhada que simula o banco de dados é protegido por mutex (pthread_mutex_t), e a coordenação da fila de tarefas entre as threads utiliza um semáforo contador (sem_t). Foram realizados experimentos comparando o processamento sequencial e paralelo com diferentes quantidades de threads, evidenciando o comportamento do pool em cenários de carga.

1 INTRODUÇÃO

Sistemas gerenciadores de banco de dados reais atendem múltiplas requisições concorrentemente: enquanto um cliente consulta um registro, outro insere um novo. O processamento paralelo, via threads, permite aproveitar os múltiplos núcleos dos processadores modernos. O desafio central é que threads de um mesmo processo compartilham a mesma memória, e o acesso simultâneo e descoordenado a estruturas compartilhadas causa condições de corrida e corrompe dados. A solução está nos mecanismos de sincronização fornecidos pelo sistema operacional: mutexes e semáforos.

Enunciado do trabalho. O professor propôs o desenvolvimento de um sistema que simule o funcionamento interno de um gerenciador de requisições a um banco de dados, com múltiplos processos e threads: um processo cliente envia requisições de consulta ou inserção via IPC; um processo servidor recebe as requisições e usa várias threads para processá-las em paralelo; e as threads utilizam mutex ou semáforo para garantir acesso seguro a uma estrutura compartilhada que simula o banco de dados. São exigidas as operações INSERT, SELECT, UPDATE e DELETE, com sintaxe conforme os exemplos do enunciado (SELECT nome WHERE id=5, INSERT id=7 nome='João'), e a entrega deve incluir código funcionando em repositório público e relatório detalhando a solução.

Os objetivos deste trabalho são: implementar comunicação real entre dois processos independentes usando IPC por meio de named pipes; processar requisições em paralelo com um pool de threads (Pthreads); proteger a estrutura compartilhada com `pthread_mutex_t` e coordenar a fila de tarefas com `sem_t`; e comparar, experimentalmente, o processamento sequencial e o paralelo.

2 DESENVOLVIMENTO

2.1 Fundamentação teórica

Processos e IPC. Processos são programas em execução com espaços de endereçamento isolados, um processo não enxerga as variáveis de outro. Para que o cliente e o servidor troquem dados, é preciso um mecanismo de comunicação entre processos (IPC) fornecido pelo sistema operacional. Neste trabalho, o IPC utilizado é o named pipe do Windows, equivalente direto do FIFO nomeado POSIX citado no enunciado: um canal de bytes com nome, mantido pelo kernel, com uma ponta em cada processo. A leitura bloqueia enquanto não há dados, ou seja, não há releitura repetida de arquivo (polling) nem memória compartilhada via variáveis globais.

Threads e Pthreads. Threads são unidades de execução dentro de um processo que compartilham a mesma memória. A biblioteca Pthreads (`pthread_create`, `pthread_join`) permite criar threads no padrão POSIX. Um pool de threads mantém um conjunto fixo de threads reutilizáveis que esperam por tarefas, evitando o custo de criar uma thread nova a cada requisição.

Mutex e exclusão mútua. Um mutex (`pthread_mutex_t`) garante que apenas uma thread por vez execute uma seção crítica, o trecho de código que acessa um recurso compartilhado. Sem exclusão mútua, duas threads inserindo simultaneamente no vetor poderiam corrompê-lo (condição de corrida).

Semáforo contador. Um semáforo (`sem_t`) é um contador atômico: `sem_wait` decrementa, bloqueando a thread quando chega a zero, e `sem_post` incrementa, acordando quem estiver esperando. No padrão produtor-consumidor, o produtor faz `sem_post` a cada item enfileirado, e cada consumidor faz `sem_wait` para retirar um item, de modo que as threads dormem enquanto não há trabalho, sem gastar CPU (sem busy waiting).

Concorrência e ordem. Processamento paralelo implica ausência de garantia de ordem entre requisições, já que elas são atendidas conforme a disponibilidade das threads. Por isso, cada requisição carrega um número identificador repetido na resposta, permitindo ao cliente casar pergunta e resposta mesmo quando chegam fora de ordem.

2.2 Arquitetura do sistema

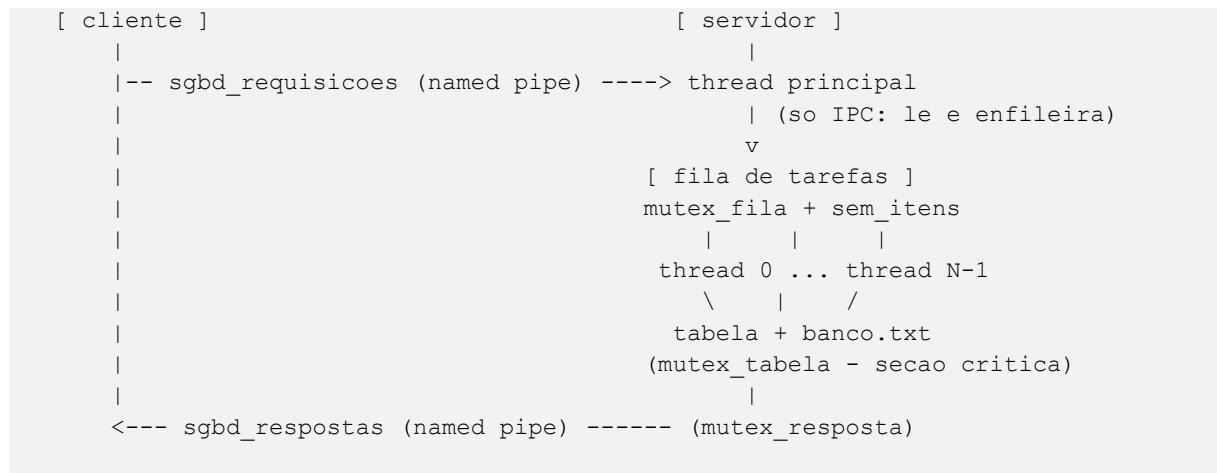


Figura 1 - Arquitetura geral do sistema cliente-servidor

O processo cliente cria o canal de respostas, ficando com a ponta de leitura, e abre o canal de requisições publicado pelo servidor. Ele opera em dois modos: sequencial (padrão), em que envia um comando, espera a resposta e só então envia o próximo, mantendo as respostas na ordem; e --paralelo, em que envia todas as requisições de uma vez e depois recolhe as respostas, que voltam fora de ordem, cada uma marcada com o número da thread que a atendeu, evidenciando o paralelismo.

No processo servidor, a thread principal faz apenas IPC: lê mensagens do canal de requisições e as enfileira. Quem interpreta comandos, acessa a tabela e escreve respostas são as threads do pool, em paralelo, de modo que uma requisição demorada nunca bloqueia a leitura do canal.

A camada compartilhada (`banco.h`) é dividida em duas partes: a Parte 1 (IPC) define o formato da mensagem, uma struct de tamanho fixo, e as funções do canal (`criarCanal`, `abrirCanal`, `aguardarConexao`, `lerMensagem`, `escreverMensagem`); a Parte 2 (`Banco`) contém a tabela compartilhada, o mutex que a protege, a persistência em `banco.txt` e o interpretador de comandos. Como os dois processos incluem o mesmo `banco.h`, ambos interpretam exatamente os mesmos bytes trocados pelo canal.

A sintaxe dos comandos segue os exemplos do enunciado: INSERT id=7 nome='João', SELECT nome WHERE id=5, UPDATE id=7 nome='Maria', DELETE WHERE id=7 e LISTAR.

2.3 Implementação

O sistema foi implementado em C++17, com IPC via named pipes do Windows (CreateNamedPipeA, ReadFile, WriteFile), threads criadas com Pthreads (pthread_create, pthread_join) e sincronização com pthread_mutex_t e sem_t. A compilação é feita por um makefile com mingw32-make, gerando os binários servidor.exe e cliente.exe como dois executáveis distintos.

Três recursos, três seções críticas. Cada recurso compartilhado do servidor tem seu próprio mutex, separados de propósito para que threads operando em recursos diferentes não se bloqueiem sem necessidade: a tabela e o banco.txt são protegidos por mutex_tabela, já que duas threads inserindo ao mesmo tempo corromperiam o vetor e duas reescrevendo o arquivo o deixariam pela metade; a fila de tarefas é protegida por mutex_fila em conjunto com o semáforo sem_itens, que conta as tarefas e acorda consumidores sem gastar CPU; e o canal de respostas é protegido por mutex_resposta, já que duas threads escrevendo juntas nele entregariam respostas intercaladas.

```
for (;;) {
    sem_wait(&sem_itens);          // dorme ate existir tarefa

    pthread_mutex_lock(&mutex_fila); // secao critica da fila
    if (fila.empty()) {             // so ocorre no encerramento
        pthread_mutex_unlock(&mutex_fila);
        break;
    }
    const Mensagem requisicao = fila.front();
    fila.pop();
    pthread_mutex_unlock(&mutex_fila); // libera a fila

    // executa o comando na tabela (mutex_tabela, dentro de executarComando)
    const std::string texto = executarComando(requisicao.texto);

    pthread_mutex_lock(&mutex_resposta); // secao critica do canal
    escreverMensagem(canal_respostas, resposta);
    pthread_mutex_unlock(&mutex_resposta);
}
```

Figura 2 - Laço principal de uma thread do pool

O semáforo cumpre o papel de avisar quando há trabalho novo: a thread dorme no `sem_wait` enquanto não há tarefas, com zero consumo de CPU, e acorda quando a thread principal faz `sem_post` logo após enfileirar uma requisição.

A fila sem limite de tamanho. A fila de tarefas não tem capacidade máxima, de propósito. Uma fila limitada permitiria um impasse em quatro pontas no modo paralelo: o cliente envia todas as requisições antes de ler respostas; se o canal de respostas enchesse, as threads do pool ficariam bloqueadas escrevendo nele; com a fila cheia, a thread principal também travaria ao enfileirar e pararia de drenar o canal de requisições, que encheria e travaria o cliente, que nunca chegaria a ler as respostas que destravariam as threads. Com a fila sempre aceitando, a thread principal nunca deixa de drenar o canal, e o impasse não chega a se formar.

O arquivo `banco.txt` é reescrito a cada INSERT, UPDATE ou DELETE dentro da mesma seção crítica da tabela: se duas threads reescrevessem o arquivo simultaneamente, ele sairia corrompido, já que o arquivo é um recurso compartilhado como o vetor. Como a gravação ocorre a cada alteração, os dados sobrevivem mesmo a um encerramento abrupto do servidor. Ao encerrar, quando o cliente desconecta, o laço de leitura da thread principal termina e é enviado um `sem_post` por thread do pool: cada uma acorda, encontra a fila vazia e sai do laço (quem ainda encontra trabalho enfileirado o processa antes de sair). O `pthread_join` garante que nenhuma thread continua viva após o fim do main.

2.4 Resultados obtidos com as simulações

Antes de medir o desempenho, o sistema foi validado funcionalmente enviando manualmente cada tipo de requisição pelo cliente interativo e conferindo a resposta devolvida pelo servidor, conforme o formato implementado em `executarComando` (`banco.h`). A Tabela 1 resume os casos testados, partindo do `banco.txt` no estado inicial (registros de id 1, 2 e 3) e usando o id=10 para os casos de inserção.

Tabela 1 - Casos de teste funcionais executados no cliente interativo

Operação	Comando enviado	Resposta obtida	Resultado
INSERT	INSERT id=10 nome='Demo'	OK inserido id=10 nome='Demo'	Registro criado
INSERT	INSERT id=10 nome='Demo'	ERRO id 10 ja existe	Duplicata rejeitada
SELECT	SELECT nome WHERE id=10	OK id=10 nome='Demo'	Registro encontrado
SELECT	SELECT nome WHERE id=99	OK 0 registros para id=99	Ausência tratada
UPDATE	UPDATE id=10 nome='Outro'	OK atualizado id=10 nome='Outro'	Registro atualizado
UPDATE	UPDATE id=99 nome='X'	ERRO id 99 nao encontrado	Erro tratado corretamente
DELETE	DELETE WHERE id=10	OK removido id=10	Registro removido

DELETE	DELETE WHERE id=10	ERRO id 10 nao encontrado	Dupla remoção tratada
LISTAR	LISTAR	OK 3 registro(s): [1,Joao Silva] [2,Maria Souza] [3,Ana Paula]	Estado final conferido

Também foi validado o comportamento sob concorrência real, usando o modo --paralelo com um lote de comandos variados: as respostas retornaram fora de ordem, cada uma identificada pelo id da requisição e pela thread que a atendeu, sem nenhum registro perdido ou duplicado, e a distribuição de carga entre as threads (detalhada na Seção 2.5.3) confirmou que o pool efetivamente reparte o trabalho. Com a corretude funcional validada, partiu-se para a medição de desempenho apresentada a seguir.

2.5 Resultados obtidos com a implementação

2.5.1 Metodologia

O cenário de teste consistiu em um lote de $N = 2000$ requisições enviado pelo cliente, variando a quantidade de threads do servidor (1, 2, 4, 8), o modo do cliente (sequencial ou --paralelo) e o tipo de operação (SELECT, somente leitura; UPDATE, escrita com regravação de banco.txt dentro da seção crítica). Antes de cada medição o banco.txt foi restaurado ao estado inicial e o servidor foi reiniciado do zero com o número de threads da configuração; o tempo medido (Measure-Command do PowerShell) cobre somente o cliente, do primeiro envio à última resposta. Cada configuração foi medida três vezes, reportando-se cada rodada e a média aritmética. Ambiente de teste: AMD Ryzen 3 3300X (4 núcleos / 8 threads lógicos), 16 GB de RAM, Windows 11 Pro, compilado com MinGW-w64 g++ -O2.

2.5.2 Resultados de desempenho

Tabela 2 - Tempo total para 2000 requisições SELECT, modo paralelo

Threads	Rodada 1 (s)	Rodada 2 (s)	Rodada 3 (s)	Média (s)
1	0,083	0,058	0,048	0,063
2	0,066	0,058	0,058	0,061
4	0,086	0,105	0,087	0,093
8	0,063	0,884*	0,048	0,332

* outlier isolado (ver Seção 2.5.4); remediações da mesma célula: 0,078 s, 0,056 s e 0,070 s; mediana das três rodadas originais: 0,063 s.

Tabela 3 - Tempo total para 2000 requisições UPDATE, modo paralelo

Threads	Rodada 1 (s)	Rodada 2 (s)	Rodada 3 (s)	Média (s)
1	1,954	2,012	2,051	2,006
2	2,317	2,822	2,227	2,455
4	3,117	6,827*	3,332	4,425

8	3,578	3,277	3,780	3,545
---	-------	-------	-------	-------

* outlier isolado (ver Seção 2.5.4); remediações: 3,405 s e 3,333 s; mediana das três rodadas originais: 3,332 s.

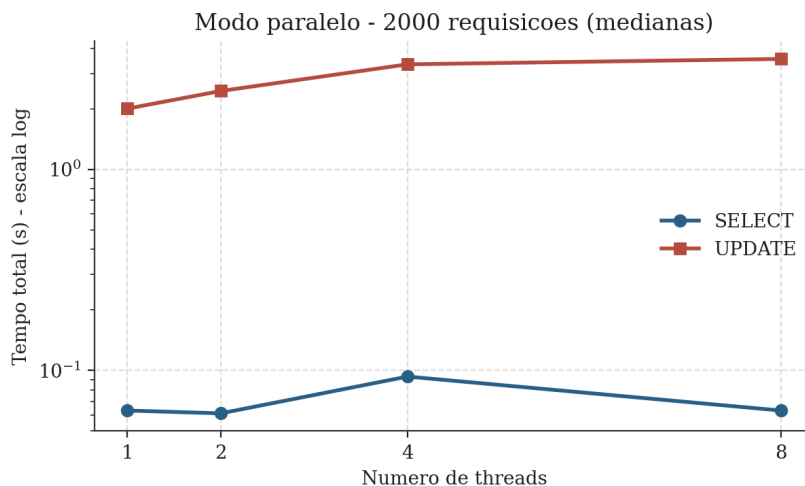


Figura 3 - Tempo total (escala log) por número de threads, modo paralelo

Tabela 4 - Tempo total para 2000 requisições SELECT, modo sequencial

Threads	Rodada 1 (s)	Rodada 2 (s)	Rodada 3 (s)	Média (s)
1	0,121	0,084	0,112	0,106
4	0,101	0,092	1,496*	0,563

* outlier isolado (ver Seção 2.5.4); remediações: 0,099 s, 0,067 s e 0,089 s; medianas: 1 thread 0,112 s, 4 threads 0,101 s (diferença menor que 10%).

Tabela 5 - Tempo total para 2000 requisições UPDATE, modo sequencial (apoio)

Threads	Rodada 1 (s)	Rodada 2 (s)	Rodada 3 (s)	Média (s)
1	4,658	2,637	3,039	3,445
2	3,962	2,708	2,820	3,163
4	3,214	2,442	2,620	2,759
8	2,806	2,084	1,962	2,284

A rodada 1 (cache frio de disco) é sistematicamente a mais lenta. Remediações com cache quente: 1 thread 2,366 s / 2,302 s contra 8 threads 2,344 s / 2,069 s, praticamente empatados, como discutido na Seção 2.6.1.

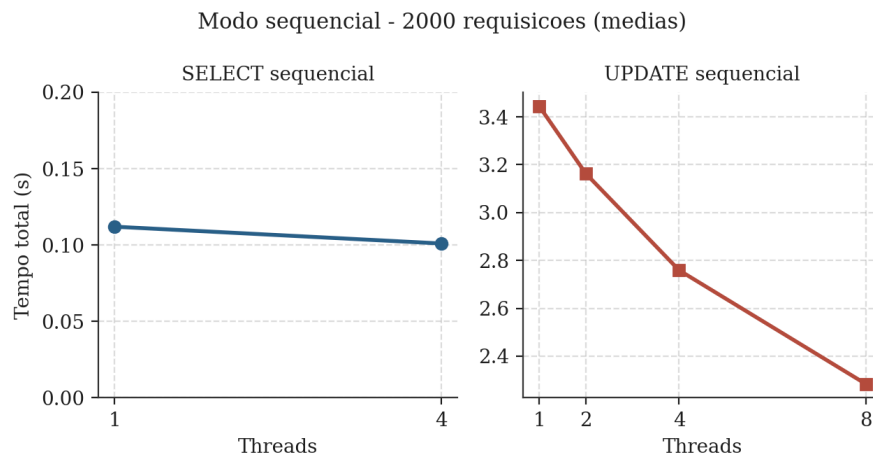


Figura 4 - Tempo total por número de threads, modo sequencial

2.5.3 Distribuição de carga entre as threads

O servidor imprime, ao encerrar, quantas requisições cada thread atendeu. Com 4 threads no modo paralelo, a distribuição observada (rodada 3) foi: SELECT — thread 0: 488, thread 1: 491, thread 2: 526, thread 3: 495, totalizando 2000; UPDATE — thread 0: 490, thread 1: 504, thread 2: 526, thread 3: 480, também totalizando 2000, com 3 registros permanecendo em banco.txt ao final. A mesma divisão equilibrada ocorre no modo sequencial (por exemplo, 500/501/499/500 em uma das medições), confirmando que o pool distribui trabalho em ambos os modos — o que muda é quantas tarefas coexistem simultaneamente na fila.

2.5.4 Nota sobre variação (outliers)

Quatro das 48 medições destoaram isoladamente da célula em que apareceram (paralelo SELECT 8 threads/rodada 2: 0,884 s; paralelo UPDATE 4 threads/rodada 2: 6,827 s; sequencial SELECT 2 threads/rodada 3: 0,409 s; sequencial SELECT 4 threads/rodada 3: 1,496 s), contra vizinhas na faixa de 0,05-0,17 s e 2,0-3,8 s. Cada uma foi remediada de 2 a 3 vezes e voltou ao patamar das demais rodadas, sem reproduzir o pico, o que atribuímos a interferência externa do sistema operacional (escalonamento, antivírus) e não ao sistema desenvolvido. Por transparência, as tabelas mantêm os valores originais e a média aritmética simples, com medianas e remediações registradas ao lado de cada outlier.

2.6 Análise e discussão

Sequencial vs. paralelo. No modo sequencial o cliente opera em ping-pong (envia, espera, envia), de modo que há no máximo uma tarefa por vez na fila e o número de threads não deveria influenciar o tempo total, o que os dados com cache quente confirmam: em SELECT, as medianas com 1 e 4 threads diferem menos de 10% (0,112 s contra 0,101 s); em UPDATE, com cache quente, 1 thread (2,37 s / 2,30 s) empata com 8 threads (2,34 s / 2,07 s). A aparente melhora da Tabela 5 (3,445 s para 2,284 s) vem quase toda da rodada 1 com cache frio, que penalizou mais as execuções longas, e não de paralelismo real — o tempo sequencial é dominado pela latência de ida e volta pelo canal, não pela falta de threads. Já no modo paralelo o lote inteiro é enfileirado de uma vez e as threads competem pelas tarefas: só aí o tamanho do pool passa a importar, e cada operação reage de um jeito diferente.

Leituras vs. escritas. A diferença de patamar entre as duas operações é de quase duas ordens de grandeza: entre 0,06 e 0,11 s para 2000 SELECTs contra 2,0 a 3,5 s para 2000 UPDATEs. A causa está no código: o SELECT segura o mutex_tabela apenas para uma busca linear em três elementos, enquanto o UPDATE reescreve o banco.txt dentro da mesma seção crítica a cada requisição, serializando 2000 escritas de arquivo no mutex e no disco. Mais threads chegam a piorar o UPDATE (de 2,01 s com 1 thread para 3,55 s com 8), porque N threads disputando o mesmo lock só adicionam contenção e trocas de contexto sem nenhum trabalho de fato sobreposto, um efeito previsto pela lei de Amdahl quando o trecho serial domina. Já o SELECT paralelo, leve demais, fica limitado pelo custo do próprio IPC e não escala: 1, 2, 4 e 8 threads empatam na faixa de 0,06 a 0,09 s. A conclusão prática é que o paralelismo acelera o que é de fato paralelizável, e a escrita serializada no mutex e no disco não é.

Escala de threads. Na máquina de teste (4 núcleos, 8 threads lógicos) não houve ganho relevante ao passar de 1-2 para 4-8 threads em nenhum cenário: o SELECT já satura o canal com 1 thread, e o UPDATE serializa no lock. Isso não é uma falha do pool — a distribuição de carga (Seção 2.5.3) mostra as 4 threads recebendo cerca de 500 requisições cada, confirmando que o mecanismo de divisão funciona; o que falta é trabalho paralelizável para dividir, dado o banco de apenas 3 registros e a seção crítica cobrindo a operação inteira. Em um cenário com comandos mais pesados, com I/O fora do lock ou travas por registro em vez de uma trava para a tabela inteira, seria esperado ganho até por volta de 4 threads, com estabilização acima disso por contenção do mutex.

Ordem de chegada das respostas. No modo paralelo, as respostas chegam fora de ordem, identificadas pelo id da requisição e pela thread que a atendeu, o que é evidência direta de execução concorrente. O custo é a ausência de garantia de ordem entre requisições diferentes, uma limitação aceita e documentada, contornada pelo id repetido na resposta e pelo uso do modo sequencial sempre que a ordem importar.

3 CONCLUSÃO

O sistema implementado atende aos requisitos do enunciado: dois processos independentes comunicando-se por IPC real (named pipes), um pool de threads Pthreads processando requisições em paralelo, e exclusão mútua protegendo a estrutura compartilhada com pthread_mutex_t nas três seções críticas identificadas (tabela e arquivo, fila de tarefas, canal de respostas) e sem_t coordenando o padrão produtor-consumidor sem consumo de CPU em espera. O encerramento é limpo, com um post por thread, join de todas e estatísticas de atendimento por thread. A escolha consciente de uma fila sem limite evita um impasse de quatro pontas no modo paralelo, uma análise de deadlock aplicada e não apenas teórica.

Os experimentos confirmaram o modelo esperado: carga dividida de forma equilibrada entre as threads, modo sequencial limitado pela latência do canal e não pelo número de threads, UPDATEs serializados pela regravação de banco.txt dentro da seção crítica (a ponto de 1 thread superar 8 no modo paralelo) e SELECTs leves demais para escalar, limitados pelo próprio custo do IPC. De modo geral, o trabalho permitiu compreender na prática os principais desafios da programação concorrente: sincronização, condições de corrida, prevenção de deadlock e os trade-offs entre diferentes mecanismos de exclusão mútua.

APÊNDICE A - EXECUÇÃO

```
mingw32-make          # compila servidor.exe e cliente.exe
servidor 4             # terminal 1 (4 threads no pool)
cliente               # terminal 2, digita comandos, Ctrl+Z + Enter
encerra
cliente comandos.txt  # lote de comandos a partir de arquivo
cliente comandos.txt --paralelo # lote em rajada (respostas fora de ordem)
```

APÊNDICE B - ROTEIRO DE DEMONSTRAÇÃO

1) Modo sequencial com comandos variados (INSERT, SELECT, UPDATE, DELETE, LISTAR), demonstrando ordem preservada. 2) Modo --paralelo com o mesmo lote, mostrando respostas fora de ordem, cada uma marcada com a thread que a atendeu. 3) Encerramento do servidor, exibindo as estatísticas de atendimento por thread.

APÊNDICE C - REPOSITÓRIO

O código-fonte completo está disponível publicamente em:

<https://github.com/naasdd/trabalho-m1-SO>